Module 2 Tangent: Error Handling

Error Handling

Not to be confused with “error ignoring”

0.

Assignment Review

Module 2 Quiz

1.

{{Title}}

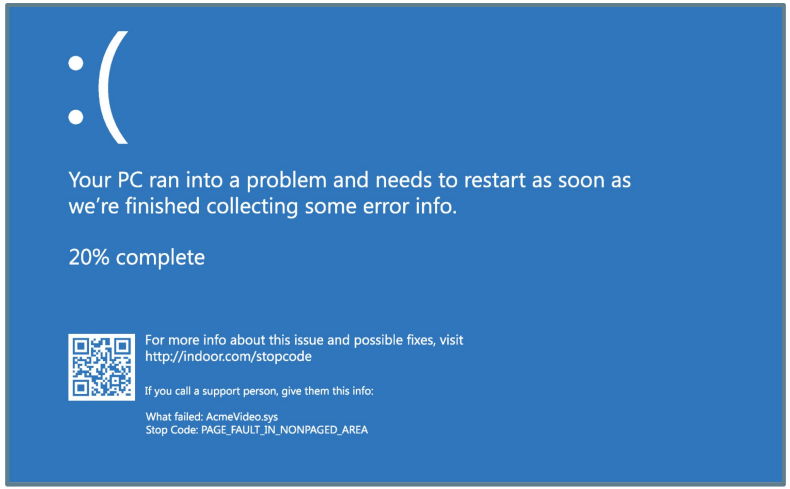
Error 404: title not found.

Errors 101

Definitions

Roughly, an error condition is a scenario where something *can* go wrong. An error is when it *has* gone wrong.

- “Wrong” could mean anything from “not right” to “critical system failure”.
  - Invalid user input (condition)
  - Network/IO failures (error)
- Errors and error conditions are a constant part of software eng.



(Rough) Categorization

Logical Errors

Occurs when an invalid operation is attempted to be performed.

- Typically has clea(er) causes and can be reproduced with the right input.
  - Broken preconditions/invariants.
  - Bad user input, e.g. forms/APIs
- Often addressed by fixing code logic, updating input data, or both.

System Errors

Occurs when a system (often external) is unable to execute a valid operation.

- Typically outside of your control and harder to reproduce, but can be predicted.
  - Network/IO issues, e.g. packet loss
  - Service availability, e.g. GitHub outage
- Often addressed by retries, redundancy, and lower-level hardware/software techniques.

Recoverability

Recoverable Errors

Errors that have predictable causes and can occur as part of “normal” execution.

- Typically involves external factors that are not fully reliable/trusted.
  - User input, e.g. form/API fields
  - External systems, e.g. Network/IO
- Can be retried, either automatically or following updated input/state.
- Best indicated at **compile-time**.

Unrecoverable Errors

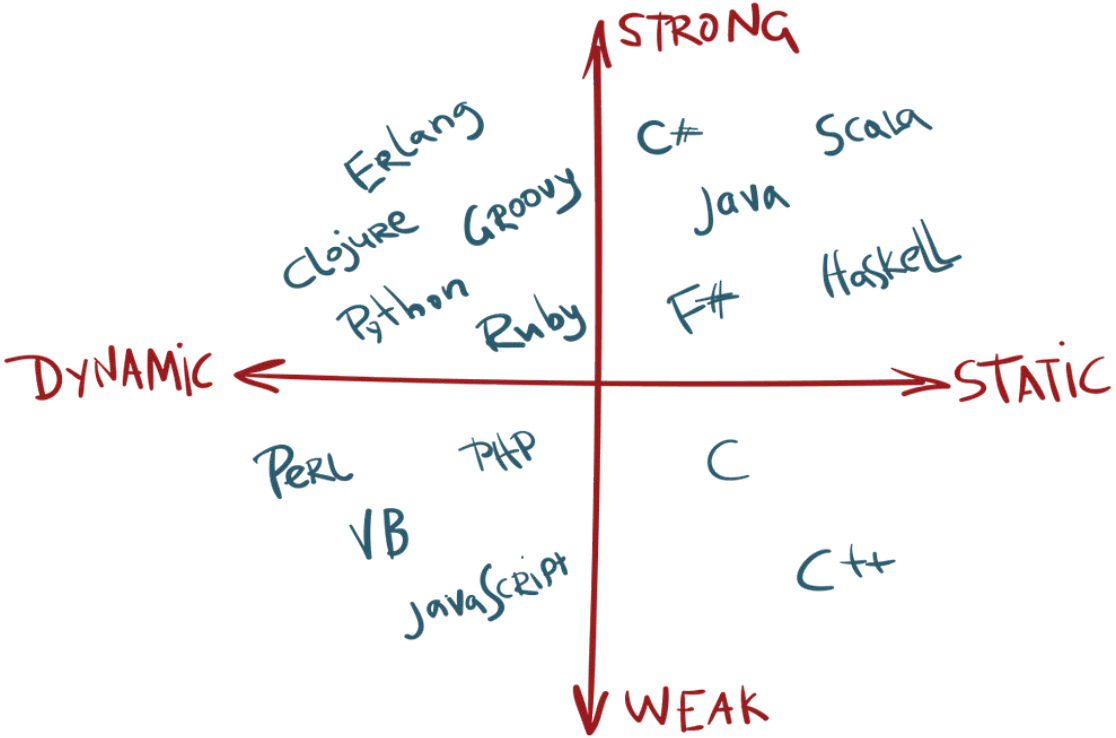
Errors that have unpredictable causes and can occur at unexpected times/places.

- Typically indicates a programming bug, either in your code or external assumptions.
  - Broken preconditions/invariants.
  - Bugs in libraries, external services, etc.
- Risky to resume - may leave the program in an invalid state causing cascading failures!
  - Try to “gracefully” restore to known state.
- Typically can’t be known until **runtime**.

Bugs are unrecoverable!

We can predict system errors and validate user input, but programming bugs are rarely recoverable.

Elixir: “Let it crash” - recover to stable state.



Static-Dynamic vs Strong-Weak Typing

2.

Error Models

More on this Wednesday; for now - Exceptions!

Error Models

Definitions

An error model describes a style of handling errors for a function/system.

- Programming languages typically have a “preferred” error model.
- In practice, multiple models are used depending on the scenario.
  - Each error model has it’s own advantanges/disadvantages!

Tradeoffs

Each error model has their own tradeoffs. Some criteria for evaluating these:

- Safety: Is the error part of the type system, and/or will the language enforce handling
- Separation: To what extent is the error “separated” from normal operations?
  - Too much, and errors can be ignored.
  - Too little, and they’re hard to work with.
- Composition: How seamlessly can errors be composed with other parts of a program?

Suggested Reading:

Midori: The Error Model

~Joe Duffy (Pulumi, Microsoft)

Sentinel Values	Error Codes (and values)	Multiple Returns
Sum Types	Result Types (and Monads)	Exceptions (checked/unchecked)

Error Models

Exceptions

Definition

An exception is a method of error handling that interrupts control flow.

- Exceptions are thrown (raised) and “travel” up the call stack until caught.
  - Roughly like a second return value.
- Exceptions are generally expensive compared to normal control flow.
  - Must find catch/finally blocks.
  - Often creates a stack trace (debug).

Checked & Unchecked

Java is the only major language to support checked exceptions for static validation.

- Checked exceptions are part of the function declaration and **must** be caught/rethrown.
  - Best for errors that are recoverable.
  - May require updating multiple functions.
  - Java: Integrates poorly with lambdas.
- Unchecked exceptions do not need to be declared and can be thrown from anywhere.
  - Best for programming errors (bugs).

```java
String string = …;

try {
  int n = Integer.parseInt(string);
  //success!
} catch (NumberFormatException e) {
  //failure
}

//concern: this compiles???
Integer.parseInt(string);
//java.lang.NumberFormatException
```

```java
String string = …;

try {
  var tokens = Lexer.lex(string);
  //success!
} catch (LexException e) {
  //failure
}

//advantage: this errors!
Lexer.lex(string);
//(no runtime error - checked!)

//concern: painful with lambdas
Stream.of(string).map(Lexer::lex);
```

Java: Unchecked vs Checked Exceptions

Module 2 Tangent: Error Handling

Error Handling

Not to be confused with “error ignoring”

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals